

Report
Of
FULL STACK WEB DEVELOPMENT

Submitted
To
School of Computer Science and Engineering
IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of
B.Tech CSE



By
Roll Number of Student: 25SCS1003004007
Name of Student: Vishal Mukesh Sharma
Batch: 2025 - 2029

CANDIDATE'S DECLARATION

I, Vishal Mukesh Sharma, do hereby declare that the internship report titled “Full Stack Web Development” has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in CSE. All the references have been quoted and I have not taken as such any material from any other source. I affirm that this report is my own and has not been submitted to any other institute for any degree/diploma requirement and further I shall be solely responsible for any kind of copyright violation in this regard.

This report documents the work carried out by me as the Core Full Stack Developer of Edurack, an EdTech startup that I co-founded and helped build and launch at www.edurack.in.

Signature

Student Name : Vishal Mukesh Sharma

Roll No. : 25SCS1003004007

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported and guided me throughout my internship at Edurack. This internship gave me an opportunity to gain practical, hands-on experience in full stack web development and to apply my technical skills to a real, production platform used by real students and mentors.

I am thankful to my co-founders, Tarun Yadav (Co-Founder & Chief Technology Officer) and Archita Priyadarshinee (Co-Founder), for their guidance, constructive feedback, and support during the internship. Working alongside them helped me understand practical approaches to building a multi-role, production-grade web application and improved my programming, architectural, and problem-solving skills.

During the internship, I worked on the core engineering of Edurack, including the CBT (Computer-Based Test) exam simulator, the multi-role authentication and access-control system, the Razorpay-based payments and monetization system, and the mentor, intern, promoter, and admin portals. Working on these systems gave me practical exposure to full stack architecture, database design, secure authentication, payment integration, cloud storage, and production deployment.

I would also like to express my gratitude to my teachers and academic mentors at IILM University for their continuous support and encouragement throughout the internship. Their guidance helped me maintain a systematic approach towards learning and completing the assigned project work.

Finally, I express my heartfelt gratitude to my parents and family for their constant encouragement, support, and blessings throughout this internship and the journey of building Edurack. Their motivation helped me stay focused and complete the internship work successfully.

Signature

Student Name : Vishal Mukesh Sharma

Roll No. : 25SCS1003004007

TABLE OF CONTENTS

1. Candidate's Declaration	i
2. Acknowledgement	ii
3. Internship Completion Certificate	iii
3.1. Internship Appointment Letter	iv
4. Project Description	1
4.1. Introduction	1
4.2. Organization Profile	2
4.3. Problem Statement	3
4.4. Project Objectives	4
4.5. Scope of the Project	5
4.6. Technologies and Tools Used	6
4.7. System Architecture	8
4.8. Methodology	10
4.9. Expected Outcomes	12
4.10. Certificate and Communication Proof	13
5. Bibliography / References	14

3. Internship Completion Certificate



3.1. Internship Appointment Letter

Date: 10 June 2026

TO,
Vishal Mukesh Sharma
Roll No.: 25SCS1003004007

Subject: Appointment Letter for Internship in Full Stack Web Development

Dear Vishal,

On behalf of the founding team, we are pleased to formally engage you as an intern at Edurack in the capacity described below. As a co-founder of the company, your appointment as intern is issued for the purpose of academic documentation of the engineering work carried out by you on the platform, and reflects the actual scope of work performed.

During this tenure, you have worked on the core product — the CBT exam simulator, authentication systems, payments integration, and the mentor/intern/promoter/admin portals — under the direct oversight of the CTO. Your progress has been reviewed on an ongoing basis through code review and product testing.

Internship Role	Intern – Core Full Stack Developer
Commencement Date	10 June 2026
Completion Date	10 August 2026
Work Mode	Remote / Founding Team – Project-Based

We look forward to a mutually beneficial learning journey and a successful platform launch.

Tarun Yadav
Co-Founder & Chief Technology Officer, Edurack

4. Project Description

The internship provided practical, production-level exposure to full stack web development through direct, hands-on engineering work on Edurack — a live EdTech platform for NEET, JEE, CUET and IPMAT aspirants, launched publicly on 10 September 2026 at www.edurack.in. Unlike a task-based internship at an external organisation, this internship involved end-to-end ownership of core product systems as the startup's Core Full Stack Developer, working directly with the Co-Founder & CTO.

The work progressed from designing the exam-taking engine and multi-role authentication system to integrating payments, cloud media delivery, and building the mentor, intern, promoter, and admin-facing portals that make up the full platform.

The systems built and owned during the internship were:

1. CBT (Computer-Based Test) Exam Simulator and Test Engine
2. Multi-Role Authentication and Access-Control System
3. Payments, Purchases and Monetization System (Razorpay Integration)
4. Mentor, Intern, Promoter and Admin Portals

These systems together cover frontend engineering, backend server-function design, database modelling, authentication and security, third-party payment integration, cloud storage and video delivery, and production deployment. The project work also provided practical experience in system design, API design, testing, debugging, and shipping features to real users.

4.1. Introduction

Full stack web development requires the ability to design and build systems that work reliably across the client, the server, and the database, while also integrating external services such as payment gateways, authentication providers, and cloud storage. With the increasing reliance of students on digital exam-preparation tools, building a platform that is both realistic and trustworthy — accurate mock tests, verified mentors, and secure payments — has become an important engineering problem.

The internship provided an opportunity to design and implement a production web application end-to-end. Instead of isolated academic exercises, the assigned work required building working systems that are live today and used by real students and mentors on www.edurack.in.

The first major system was the CBT Exam Simulator and Test Engine, a pixel-accurate replica of the NTA's official computer-based test interface, including question navigation, the answer palette, section switching, timers, and server-side grading.

The second was the Multi-Role Authentication and Access-Control System, which manages four distinct classes of users — students, mentors, promoters, and interns — each with an isolated authentication flow, session mechanism, and permission model, alongside a separate Firebase-based admin authentication layer.

The third was the Payments, Purchases and Monetization System, built on Razorpay, which handles order creation, signature verification, commission splitting between the platform and mentors, and the unlocking of paywalled test bundles and mentorship sessions.

The fourth was the set of Mentor, Intern, Promoter and Admin Portals — dashboards through which mentors manage batches, sessions and earnings; interns receive and submit question-writing tasks; promoters track referrals; and admins oversee the entire platform.

Together, these systems provided practical exposure to full stack architecture, secure authentication, database design, third-party API integration, cloud infrastructure, and the discipline of shipping and maintaining a live, revenue-generating product.

The internship also helped in improving system design thinking, API design, testing, debugging, and problem-solving skills, along with a much deeper appreciation for the operational and security responsibilities that come with running a production application handling real user data and real payments.

4.2. Organization Profile

The internship was undertaken at Edurack, an EdTech startup that I co-founded along with Tarun Yadav (Co-Founder & CTO) and Archita Priyadarshinee (Co-Founder). Edurack is an independent web platform dedicated to pixel-exact Computer-Based Test (CBT) simulators and structured mentorship for students preparing for NEET, JEE, CUET and IPMAT.

The platform was publicly launched on 10 September 2026 and is live at www.edurack.in. It allows students to take realistic mock tests on an interface modelled closely on the official NTA CBT screen, receive detailed subject-wise and topic-wise performance analytics, and connect with mentors who have themselves cleared these examinations. Mentors on the platform run their own batches, set their own pricing, and are supported by Edurack's payments, scheduling and discovery infrastructure. The platform also runs an internship program of its own — the Question Ingestion Internship — through which interns help write and review exam questions.

As Core Full Stack Developer and a co-founder, I was responsible for the end-to-end engineering of the platform: architecting the database, building the exam engine, implementing authentication and payments, and developing the mentor, intern, promoter and admin portals, in direct collaboration with the CTO.

The internship involved working across the full stack — frontend interfaces, backend server functions, database schema design, third-party integrations (Razorpay, Firebase, AWS S3/CloudFront, Resend, Supabase) — and following structured development practices such as requirement analysis, iterative implementation, code review, testing, and staged production deployment.

4.3. Problem Statement

Students preparing for competitive exams such as NEET, JEE, CUET and IPMAT often practise on mock-test interfaces that look nothing like the actual exam-day screen, which increases the chances of losing time and marks to unfamiliarity rather than lack of preparation. At the same time, students rarely have structured access to mentors who have actually cleared the exam they are preparing for, and existing mentorship options are often unstructured, expensive, or hard to trust. Building a platform to solve this required addressing several concrete engineering problems.

The first problem was building an exam interface that is exam-accurate down to the palette colours, section switching, marking scheme display and navigation, while keeping grading logic — including the correct answers and solutions — strictly server-side so that no answer key can ever be inspected or extracted from the client before submission.

The second problem was supporting several very different classes of users — students, mentors, promoters and interns — each needing its own sign-up flow, session model and permission boundaries, without one role's authentication logic leaking into or being reusable by another, while keeping the codebase maintainable.

The third problem was monetization: safely taking real payments for test bundles and mentorship sessions, splitting revenue between the platform and individual mentors, and ensuring that access to paid content is unlocked only after a payment has been cryptographically verified rather than trusted from the client.

The fourth problem was giving mentors, interns, promoters and admins working tools of their own — batch and session management, task assignment and review, referral tracking, and platform-wide oversight — built on top of the same underlying data model as the student-facing product.

A common challenge across all four systems was that a task-based, security-conscious approach had to be followed throughout — re-verifying ownership and access on the server for every sensitive action, hashing and never storing secrets or passwords in plaintext, and testing thoroughly before every feature reached real users on a live platform.

4.4. Project Objectives

The main objective of the internship was to gain practical, production-grade experience in full stack web development by taking ownership of core systems on a live EdTech platform. The major objectives of the project work were:

1. To understand and apply the principles of full stack architecture across client, server, and database layers.
2. To design and build a CBT Exam Simulator that faithfully replicates the NTA's official computer-based test interface, including timers, the question palette, and section-wise navigation.
3. To implement a server-side test engine that enforces access control before serving questions and computes grading only after submission, without ever exposing answer keys prematurely.
4. To design a multi-role authentication and access-control system covering students (Firebase Auth), mentors, promoters, and interns (isolated HMAC session-token systems), and admins.
5. To implement secure password handling using salted hashing (bcrypt) and one-time-password (OTP) verification using HMAC digests rather than plaintext storage.
6. To integrate the Razorpay payment gateway for order creation, cryptographic payment-signature verification, and commission-based revenue splitting between the platform and mentors.
7. To build mentor, intern, promoter and admin portals for batch management, session scheduling, task assignment, earnings tracking, and platform oversight.
8. To integrate cloud infrastructure — AWS S3 and CloudFront for signed, time-limited video delivery, and Supabase storage for image uploads.
9. To integrate transactional email (via Resend) for OTPs, password resets, purchase confirmations, and session-booking notifications.
10. To design and normalise a MongoDB data model that supports test bundles, purchases, mentor batches, sessions, and internship tasks.
11. To improve practical skills in system design, API design, testing, debugging, and secure coding through real production work.
12. To understand the operational responsibilities of running a live, revenue-generating web application used by real students and mentors.
13. To gain practical experience translating product and business requirements into working, deployed software.

4.5. Scope of the Project

The scope of the internship covers the end-to-end engineering of Edurack, spanning four major systems, each addressing a different part of the platform. Together, they provide practical exposure to exam-engine design, authentication and security, payments, and multi-role portal development.

The CBT Exam Simulator and Test Engine covers the exam-taking interface (question navigation, palette, timer, section switching), server-side access verification against a student's purchases before a test is served, and server-side grading and result generation after submission, for tests sourced from either the admin-curated Test Series catalogue or standalone mentor-listed Sold Tests.

The Multi-Role Authentication and Access-Control System covers sign-up, sign-in, session issuance and verification for four separate user classes — students (via Firebase Authentication), and mentors, promoters and interns (via isolated, self-issued HMAC session-token systems with bcrypt-hashed passwords) — along with a Firebase-based admin authentication layer, OTP-based email verification, and password-reset flows.

The Payments, Purchases and Monetization System covers Razorpay order creation with server-side price lookup, HMAC-based payment-signature verification, the purchases collection that unlocks paid content platform-wide, and commission-splitting logic for mentor test sales, question-ingestion payouts, and mentorship-session bookings.

The Mentor, Intern, Promoter and Admin Portals cover batch and test-series creation, session scheduling and earnings for mentors; task assignment, question drafting and submission for interns; referral and batch-selling tools for promoters; and platform-wide administration — including intern management, question review, and catalogue curation — for admins.

From a technical perspective, the overall scope includes frontend UI engineering, server-function API design, MongoDB schema design, third-party payment and authentication integration, cloud media storage and delivery, transactional email, testing, and production deployment.

The platform also provides a foundation for future improvements — additional exam types, richer analytics, adaptive testing, expanded mentor tooling, and further security hardening — as Edurack continues to grow beyond the internship period.

4.6. Technologies and Tools Used

The development of Edurack involved a modern full stack TypeScript toolchain, chosen according to the requirements of each system — realistic UI rendering for the exam engine, secure server-side logic for authentication and payments, and reliable cloud infrastructure for storage and delivery.

4.6.1. Frontend Technologies

React (with TypeScript) was used to build all user-facing interfaces, including the CBT simulator, student dashboards, and the mentor, intern, promoter and admin portals, using function components and hooks throughout.

TanStack Start and TanStack Router were used as the full stack React framework and file-based router, powering both client-side routing (e.g. `test.$testId`, `mentor-profile.$mentorId`, `course.$kind.$id`) and colocated server functions.

Tailwind CSS (v4) was used for styling, along with `tw-animate-css` for animation utilities and a `shadcn/ui`-style component library for consistent, themeable UI primitives, including light/dark theming via a custom theme provider.

TypeScript was used across the entire codebase — frontend components, server functions, and shared types (e.g. `intern-types.ts`, `promoter-types.ts`) — to keep the client and server contractually in sync.

4.6.2. Backend Technologies

TanStack Start server functions (`createServerFn`) were used to implement all backend logic — test-taking and grading, authentication, payments, portal APIs — colocated with the frontend but executed strictly server-side.

Node.js's built-in `crypto` module was used for cryptographically secure OTP generation, HMAC-based OTP and token hashing, `scrypt`-based password hashing for mentor/promoter/intern accounts, and Razorpay payment-signature verification.

Firebase Admin SDK was used for student authentication (ID-token verification via `adminAuth.verifyIdToken`) and for the separate admin authentication layer.

4.6.3. Database

MongoDB (hosted on MongoDB Atlas) was used as the primary database, accessed through the official MongoDB Node.js driver with a serverless-safe singleton connection pattern. Collections were designed for test cores and sold tests, purchases, users, mentors, promoters, interns and intern tasks, bundles, sessions, and tickets, with `ObjectId`-based referential lookups and ownership checks enforced on every server function.

4.6.4. Payments, Authentication and Security

Razorpay was integrated for order creation and payment capture, with order amounts always looked up server-side (never trusted from the client) and every payment verified using an HMAC-SHA256 signature check before a purchase is recorded.

Passwords for mentor, promoter and intern accounts are hashed using `scrypt` with a random per-user salt; OTPs and password-reset tokens are never stored in plaintext, only as HMAC-SHA256 digests keyed with a server-only secret ("pepper").

Session tokens for mentors, promoters and interns are stateless, self-issued HMAC tokens, isolated per role with separate session secrets, while students and admins authenticate via Firebase Auth.

Server-side ownership and access re-verification is applied consistently — for example, a task ID or draft ID in a request is never trusted to belong to the caller without a database check, the same rule test access and purchase checks follow.

4.6.5. Cloud Infrastructure and Third-Party Services

AWS S3 and CloudFront were used for lecture video storage and delivery, with time-limited (4-hour) pre-signed URLs generated via the AWS SDK's S3 request presigner so that video content is never publicly and permanently accessible.

Supabase Storage was used for image uploads such as mentor profile photos and question images.

Resend was used for all transactional email — OTP verification, password resets, purchase confirmations, and session-booking notifications — sent through shared, branded HTML email templates.

4.6.6. Development and Version Control Tools

Visual Studio Code was used as the primary development environment for writing and managing the project's source code.

Git was used for version control and GitHub for remote repository management and collaboration with the co-founding team.

Environment configuration was centrally validated at boot (`validateServerEnv`) to fail fast and loudly if required secrets — database URIs, API keys, AWS credentials — were missing, rather than surfacing as unexplained runtime errors in production.

4.7. System Architecture

The system architecture of Edurack consists of the four core systems developed during the internship, working together over a shared MongoDB data model. Although each system addresses a different concern, they follow a consistent layered approach in which user interaction, server-function logic, security enforcement, third-party services, and data storage work together.

4.7.1. User Interface Layer

The user interface layer is built with React, TanStack Router, and Tailwind CSS, and is responsible for accepting user input and rendering results across the different user surfaces of the platform. The CBT simulator renders questions, the answer palette, timers and section tabs; the student dashboard renders performance analytics; and the mentor, intern, promoter and admin portals each render role-specific dashboards and workflows.

4.7.2. Application and Security Processing Layer

The application and security processing layer is implemented through TanStack Start server functions. In the Test Engine, this layer re-verifies a student's purchase before serving questions and withholds correct answers and solutions until after submission. In the Authentication system, it verifies Firebase ID tokens for students and admins, and verifies HMAC session tokens and script password hashes for mentors, promoters and interns. In the Payments system, it looks up prices server-side, creates Razorpay orders, and verifies payment signatures before writing to the purchases collection. In the Portals, it re-checks that every task, draft, batch or session belongs to the caller before allowing it to be read or modified.

4.7.3. Database Layer

The database layer, backed by MongoDB Atlas, stores all persistent application data — test cores and sold tests, purchases, user/mentor/promoter/intern records, intern tasks, bundles, sessions and tickets. The application communicates with the database exclusively through server functions, never exposing direct database access to the client, and passwords are never stored in their original form.

4.7.4. Third-Party Services Layer

This layer integrates Razorpay for payments, Firebase for student/admin authentication, AWS S3/CloudFront for signed video delivery, Supabase for image storage, and Resend for transactional email. Each integration is isolated behind its own server-only module (payments.ts, firebase-admin, video-signer.ts, mailer.ts) so that credentials and provider-specific logic never reach the client bundle.

4.7.5. System-Level Architecture

The architecture of the four core systems can be summarised as follows:

CBT Exam Simulator and Test Engine:

Student → Purchase/Access Check → Question Delivery (no answers) → Test Submission → Server-Side Grading → Result & Analytics

Multi-Role Authentication:

User (Student / Mentor / Promoter / Intern / Admin) → Role-Specific Sign-In → Firebase Token or HMAC Session Token → Verified Identity → Role-Scoped Access

Payments and Purchases:

User → Checkout Request → Server-Side Price Lookup → Razorpay Order → Payment → Signature Verification → Purchase Record → Content Unlocked

Mentor / Intern / Promoter / Admin Portals:

Role User → Portal Dashboard → Server Function → Ownership Check → MongoDB Read/Write → Updated Dashboard

This layered architecture separates user interaction, application logic, security enforcement, third-party integration, and data storage, making each system easier to understand, test, secure, and extend independently as Edurack continues to grow.

4.8. Methodology

The systems built during the internship followed a structured, iterative development approach suited to a live startup environment: understanding real product requirements, designing the data model and API surface, implementing and securing the feature, testing it thoroughly, and shipping it to production, with the CTO reviewing design and code throughout.

4.8.1. Requirement Analysis

Each system began with understanding the real product requirement it needed to satisfy — for example, that the CBT simulator had to feel identical to the official NTA interface, or that mentor payouts needed accurate commission splitting. Expected inputs, outputs, and edge cases (e.g. a test purchased via a bundle vs. purchased standalone) were identified before implementation began.

4.8.2. System Planning and Design

After understanding the requirements, the data model and server-function API for each system were planned — deciding what MongoDB collections and fields were needed, which operations required server-side verification, and how each portal's UI would consume the underlying API.

4.8.3. Application Development

Each system was implemented feature by feature. The Test Engine was built to source tests from either the Test Series catalogue or standalone Sold Tests, and to strip answer data from anything sent to the client before submission. The authentication systems were built per role, mirroring a common pattern (invite/sign-up → hashed credentials → session issuance) while keeping each role's session secret and collection fully isolated.

4.8.4. Security Feature Implementation

Security was treated as a first-class requirement rather than an afterthought. Passwords were hashed with scrypt and a random salt; OTPs and reset tokens were hashed with HMAC-SHA256 and never stored in plaintext; Razorpay payments were verified with signature checks before any purchase was recorded; and every server function that reads or writes a specific record re-checks that the record belongs to the authenticated caller.

4.8.5. Data and Database Integration

MongoDB collections were designed and connected for every system that required persistent data — test cores, sold tests, purchases, user/mentor/promoter/intern records, and intern tasks — using a serverless-safe singleton connection pattern so the same database handle is reused efficiently across requests.

4.8.6. Third-Party Integration

Razorpay was integrated for payments, Firebase for student and admin authentication, AWS S3/CloudFront for signed video URLs, Supabase for image storage, and Resend for transactional email, with each integration isolated in its own server-only module and validated at application boot.

4.8.7. Testing and Debugging

After implementing each feature, it was tested with a range of inputs and conditions — valid and invalid purchases, expired sessions, incorrect payment signatures, concurrent bookings — to verify that the implemented functionality behaved correctly and securely. Errors found during testing were analyzed and corrected, and testing was repeated after every change to ensure other working features were not affected.

4.8.8. Security Testing

Security-related functionality was specifically checked during development — confirming that answer keys were never present in the network response before test submission, that paywalled content stayed locked without a verified purchase, and that one role's session token could not be used to access another role's portal.

4.8.9. Version Control and Project Management

Git was used to track changes throughout development, with GitHub used for remote repository management and collaboration with the co-founding team. Feature work was organised so that changes could be reviewed and tracked in an orderly manner.

4.8.10. Final Review and Deployment

Completed features were reviewed for functionality, security, and usability before being deployed to production at www.edurack.in, culminating in the platform's public launch on 10 September 2026.

This methodology provided a systematic approach to completing the internship work and helped in understanding the practical process of designing, securing, testing, and shipping a live, production web application.

4.9. Expected Outcomes

The internship was expected to provide practical, production-grade knowledge of full stack web development and improve the ability to design and ship complete, secure systems rather than isolated exercises. The major outcomes of the internship were:

1. Development of a fully functional CBT Exam Simulator and Test Engine, faithfully replicating the NTA's computer-based test interface with server-side access control and grading.
2. Practical understanding of building exam-accurate, timed, multi-section interfaces in React.
3. Development of a Multi-Role Authentication and Access-Control System covering students, mentors, promoters, interns and admins, with isolated session models per role.
4. Practical understanding of secure credential handling — bcrypt password hashing, HMAC-based OTP/token hashing, and Firebase ID-token verification.
5. Development of a Payments, Purchases and Monetization System integrated with Razorpay, including signature verification and commission-based revenue splitting.
6. Development of Mentor, Intern, Promoter and Admin Portals for batch management, session scheduling, task workflows, referral tracking, and platform administration.
7. Practical experience integrating cloud infrastructure — AWS S3/CloudFront for signed video delivery and Supabase for image storage.
8. Practical experience integrating transactional email (Resend) for OTPs, password resets and confirmations.
9. Design of a normalised MongoDB data model supporting tests, purchases, users of four different roles, and internship tasks.
10. Improved ability to identify, analyze and resolve issues through regular testing and debugging on a live application.
11. Better understanding of secure coding practices and the operational responsibilities of running a production, revenue-generating platform.
12. Practical experience translating real product and business requirements into working, deployed software.
13. Contribution to the successful public launch of Edurack (10 September 2026) at www.edurack.in, live and serving real students and mentors today.

Overall, the expected outcome of the internship was to develop a strong, practical command of full stack web development and gain first-hand experience turning a startup's product vision into a secure, functioning, and launched software platform.

4.10. Certificate and Communication Proof

The internship was undertaken at Edurack, the EdTech startup I co-founded and helped build as Core Full Stack Developer. The internship provided an opportunity to design, build, secure and ship the core engineering systems behind a live, publicly launched product.

The internship involved working on four major systems: the CBT Exam Simulator and Test Engine, the Multi-Role Authentication and Access-Control System, the Payments and Monetization System, and the Mentor/Intern/Promoter/Admin Portals. These systems provided practical exposure to full stack architecture, secure authentication, payment integration, cloud infrastructure, and production deployment.

The internship appointment letter and completion certificate included in this report (Sections 3 and 3.1), issued by the Co-Founder & Chief Technology Officer, serve as supporting evidence of the internship engagement and the work carried out. The platform itself — live and publicly accessible at www.edurack.in — serves as direct, verifiable evidence of the completed work.

The following documents are included in this report:

- Internship Completion Certificate issued by Edurack (Section 3).
- Internship Appointment Letter issued by Edurack (Section 3.1).
- A live, publicly accessible product (www.edurack.in) evidencing the completed engineering work.

These documents, together with the live platform, provide supporting evidence of the internship, the assigned engineering work, and the completion of the internship requirements.

5. Bibliography / References

1. Edurack, Internship Appointment Letter and Completion Certificate, 2026.
2. Edurack — NEET, JEE, CUET & IPMAT Exam Prep Platform, www.edurack.in, 2026.
3. React Documentation, documentation related to components, hooks, and client-side rendering.
4. TanStack, TanStack Start and TanStack Router Documentation, documentation related to full stack React routing and server functions.
5. MongoDB, MongoDB Documentation, documentation related to the Node.js driver, schema design, and Atlas.
6. Razorpay, Razorpay API Documentation, documentation related to order creation, payments, and signature verification.
7. Firebase, Firebase Authentication Documentation, documentation related to ID-token issuance and verification.
8. Amazon Web Services, AWS S3 and CloudFront Documentation, documentation related to object storage and signed URL generation.
9. Supabase, Supabase Storage Documentation, documentation related to file and image storage.
10. Resend, Resend Email API Documentation, documentation related to transactional email delivery.
11. Tailwind Labs, Tailwind CSS Documentation, documentation related to utility-first styling.
12. Node.js, Node.js crypto Module Documentation, documentation related to HMAC, scrypt, and secure random generation.
13. OWASP Foundation, OWASP Web Security Documentation, resources related to authentication, session management, and payment security.
14. Git, Git Documentation, documentation related to distributed version control.
15. GitHub, GitHub Documentation, documentation related to repositories and collaborative workflows.